



Private Emotional Wellness Companion — Full Architecture, Data Models, Component Design, API Reference, and Design Decisions

Version 1.1.9 · Build 97 Web · iOS · Android May 2026

CONTENTS

1	System Overview & Philosophy	9	Theme System & Color Palette
2	Technology Stack	10	Navigation Architecture
3	High-Level Architecture Diagram	11	AI / Psychological Engine
4	Data Models & Class Diagrams	12	Payment & Licensing Architecture
5	State Management	13	Feature Architecture — Chat
6	API Reference	14	Feature Architecture — Trends
7	Database Schema	15	Performance & Production Patterns
8	Screen Component Hierarchy	16	Key Design Decisions

1. System Overview & Philosophy

Imotara is a private emotional wellness companion available on iOS, Android, and web. It combines a psychologically-informed conversational AI with mood tracking, growth analytics, and a deeply personal companion relationship — all built on a privacy-first, local-first architecture.

1.1 Core Design Principles

Local-First

All conversations and data stored on-device by default. Cloud backup is explicit opt-in only. No data leaves the device without user action.

Psychological Depth

71 research-backed psychological tools (CBT, polyvagal, narrative therapy, attachment theory) applied via a 7-tier priority framework.

Multilingual

22 languages with native-script support. All features — AI responses, letters, mindset analysis, settings search — work in every language.

1.2 Platform Summary

Platform	Framework	Deployment	Version
Web	Next.js 16 (App Router)	Vercel	1.1.9
iOS	React Native (Expo SDK 54)	App Store	1.1.9 / build 97
Android	React Native (Expo SDK 54)	Play Store	1.1.9 / versionCode 97
Backend API	Next.js Route Handlers	Vercel Edge + Node	—
Database	Supabase (PostgreSQL)	Supabase Cloud	—
AI Model	GPT-4.1 (OpenAI)	OpenAI API	gpt-4.1

2. Technology Stack

React Native / Expo

SDK 54, New Architecture enabled, Hermes JIT engine, EAS Build & Submit

Next.js 16

App Router, Server Components, Route Handlers, Turbopack, static + dynamic rendering

Supabase

PostgreSQL DB, Row-Level Security, Auth (Google + Apple OAuth), Realtime, Storage

OpenAI GPT-4.1

Primary AI model for all chat replies, mindset analysis, companion letters, settings search

Azure Neural TTS

Text-to-speech for 22 languages. 42 pre-generated preview MP3s cached on CDN.

Razorpay

Android payments. Order intent → checkout → webhook → license grant flow.

Apple IAP (expo-iap)

AsyncStorage

Tailwind CSS

iOS subscriptions + token packs via StoreKit 2. Verified server-side with Apple JWS.

Mobile local persistence for all settings, history, letters, emotional arc, pending insights.

Web styling with dark: class modifiers. Mobile uses inline StyleSheet + ThemeContext.

3. High-Level Architecture



LOCAL STORAGE (Mobile - AsyncStorage):

Conversations · Threads · Settings · Letters · Journal · Arc · Pending Insights

LOCAL STORAGE (Web - localStorage):

ToneContext · Letters · Arc · Feature flags · Session state

4. Data Models & Class Diagrams

4.1 Core Message & Thread Model

HistoryItem

```
id: string
text: string
from: "user" | "bot"
timestamp: number
emotion?: string
intensity?: number
isSynced?: boolean
threadId?: string
moodHint?: string
source?: "cloud"|"local"
```

ConversationThread

```
id: string
title: string
createdAt: number

messages: HistoryItem[]
DEFAULT_THREAD_ID = "default"
```

CompanionLetter

```
id: string
generatedAt: number
body: string
companionName: string
reaction?: string
reply?: string
replyAt?: number

Archive: up to 24 letters
```

EmotionalArc

```
id: string
generatedAt: number
periodLabel: string
narrative: string

Cadence: configurable (default 30d)
```

4.2 Settings / Profile Model

ToneContext (User Profile)

```
user:
  name: string
  ageRange: string
  gender: string
  preferredLang: string

companion:
  enabled: boolean
  name: string
  relationship: string
  gender: string
  ageRange: string
```

SettingsContextValue

```
emotionInsightsEnabled: bool
companionReactionsEnabled: bool
featureTipsEnabled: bool
showSyncBadge: bool
analysisMode: "local"|"cloud"
toneContext: ToneContext
teenMode: boolean
autoSyncDelaySeconds: number
lastSyncAt: number | null
```

License (Supabase)

```
user_id: uuid
tier: "free"|"plus"|"pro"|"ent"
status: "valid"|"expired"
```

```
token_balance: integer
expires_at: timestampz
source: "apple"|"razorpay"|"webhook"
updated_at: timestampz
```

4.3 Pending Insights Bridge Model

InsightPayload (AsyncStorage Bridge)

```
weeklyRecap?: string
collectivePulse?: { heavyPercent: number }
milestone?: { id: string; themeName: string }
companionInsight?: { variant, title, body }
```

```
savePendingInsight(key, value)
clearPendingInsight(key)
getBadgeCount(): number
```

FeatureTip

```
id: string
emoji: string
title: string
tip: string
category: "chat"|"voice"|"growth"|"companion"|"privacy"|"settings"
```

```
Total: 194 tips
Rotation: 30 min active use
getCurrentTip(): Promise<{tip,index}>
flushActiveTime(): Promise<{advanced}>
```

5. State Management

```
APP ROOT (App.tsx / _app.tsx)
|
├─ ThemeContext          isDark, colors (DARK|LIGHT palette), themeMode
|   └─ useTheme()        → { colors, isDark }
|   └─ useColors()       → ColorPalette
|
├─ SettingsProvider      AsyncStorage-persisted app settings + ToneContext
|   └─ useSettings()     → SettingsContextValue (all toggles, toneContext, sync state)
|
├─ AuthContext           Supabase Auth session, Google/Apple sign-in
|   └─ useAuth()         → { accessToken, signOut, signInWithGoogle, signInWithApple }
|
└─ HistoryProvider       In-memory conversation store + sync engine
```

```

└─ useHistoryStore() → { history, addToHistory, runSync, hasUnsyncedChanges }
└─ NavigationContainer  React Navigation bottom tabs
    └─ ChatScreen
    └─ HistoryScreen
    └─ TrendsScreen
    └─ SettingsScreen

```

5.1 Data Flow — Sending a Message

- 1 User types or speaks → ChatScreen captures input
- 2 Message appended to history via `addToHistory()` → local `AsyncStorage` immediately
- 3 If `analysisMode === "cloud"`: POST to `/api/chat-reply` with Bearer token + conversation context
- 4 If `analysisMode === "local"`: `buildLocalReply()` runs entirely on-device (no network)
- 5 Bot response appended to history, rendered in `FlatList`
- 6 Background sync: `runSync()` pushes unsynced records to `/api/history/sync` if signed in

6. API Reference

Endpoint	Method	Auth	Purpose
<code>/api/chat-reply</code>	POST	Bearer / Cookie	Main AI response — applies 71 psychological tools, 22 languages
<code>/api/respond</code>	POST	Cookie	Web-specific AI response (used by companion letter, arc generation)
<code>/api/analyze</code>	POST	Cookie	Emotion analysis of messages — returns dominant emotion + intensity
<code>/api/mindset-analysis</code>	POST	Bearer / Cookie	Deep psychological analysis of conversation batch (schema, SDT, PTG)
<code>/api/settings-search</code>	POST	None	AI-powered settings finder — returns setting IDs for natural-language query
<code>/api/voice/transcribe</code>	POST	None	Whisper STT — audio → text, 55s timeout, 22 language support

/api/tts	POST	None	Azure Neural TTS — text → MP3 audio buffer, gender + language aware
/api/history	GET / DELETE	Bearer / Cookie	Fetch or delete remote conversation history
/api/history/sync	POST	Bearer / Cookie	Push unsynced messages to Supabase; merge remote records
/api/profile/sync	GET / POST	Bearer / Cookie	Read or write ToneContext (companion preferences, user profile)
/api/license/status	GET	Bearer / Cookie	Current license tier, token balance, expiry — both mobile and web
/api/license/order-intent	POST	Bearer	Create Razorpay order for Android purchase
/api/license/verify-payment	POST	Bearer	Verify Razorpay payment + grant license. Polls 5× for UPI mandate capture.
/api/license/verify-apple-purchase	POST	Bearer	Verify Apple IAP JWS transaction + grant license (ES256, dsaEncoding: ieee-p1363)
/api/payments/razorpay/webhook	POST	HMAC sig	Razorpay event handler — payment.captured → grantLicense()
/api/memory	GET / POST	Bearer / Cookie	Read/write user memory items (companion remembers user details)
/api/pulse	GET	None	Anonymous collective emotional pulse — % of users carrying something heavy

7. Database Schema (Supabase PostgreSQL)

```

TABLE: licenses
  user_id      uuid PRIMARY KEY (FK → auth.users)
  tier          text CHECK IN ('free','plus','pro','enterprise')
  status       text CHECK IN ('valid','invalid','expired')
  token_balance integer DEFAULT 0
  expires_at   timestamptz NULL
  source       text (apple | razorpay | webhook | admin)
  updated_at   timestamptz DEFAULT now()

TABLE: payment_licenses
  payment_id   text PRIMARY KEY
  user_id      uuid (FK → auth.users)
  product_id   text
  tier          text
  amount_paise integer
  currency     text DEFAULT 'INR'

```

```

    created_at      timestamptz DEFAULT now()

TABLE: user_memory
  id               uuid PRIMARY KEY
  user_id          uuid (FK → auth.users)
  type             text  (identity | preference | event)
  key              text
  value            text
  confidence        float DEFAULT 0.8
  updated_at       timestamptz DEFAULT now()
  UNIQUE (user_id, type, key)

TABLE: usage_events
  id               uuid PRIMARY KEY
  user_id          uuid (FK → auth.users)
  event_type       text
  payload          jsonb
  created_at       timestamptz DEFAULT now()

TABLE: admin_license_history
  id               uuid PRIMARY KEY
  user_id          uuid
  action           text
  changed_by       text
  old_tier         text
  new_tier         text
  note             text
  created_at       timestamptz DEFAULT now()

TABLE: blog_comments
  id               uuid PRIMARY KEY
  post_slug        text
  author_name       text
  content           text
  approved          boolean DEFAULT false
  created_at       timestamptz DEFAULT now()

RLS POLICIES: All tables have Row-Level Security enabled.
- users can only read/write their own rows (user_id = auth.uid())
- service_role key bypasses RLS for admin operations

```

8. Screen & Component Hierarchy

```

RootNavigator (Bottom Tab Navigator)
├── ChatScreen (shell - useFocusEffect + double-RAF deferred mount)
│   └── ChatScreenContent (heavy - 5593 lines, ~60 hooks)
│       ├── ImotaraChatBubble      Message rendering with source icon
│       ├── ImotaraTypingIndicator  Animated dots during AI generation
│       ├── CompanionQuickPanel     Avatar + companion name header
│       ├── OpenLoopCard            Unresolved theme callback
│       ├── UnsentLetterModal       Write-to-someone-you-can't-reach
│       ├── BreathingModal          4-7-8 / box breathing exercise
│       ├── DiscoveryCard           Feature nudge (trends/companion/offline)
│       ├── UpgradeSheet            Plan upgrade (IAP iOS / Razorpay Android)
│       └── OnboardingModal         First-run companion setup
└── HistoryScreen (shell - useFocusEffect + double-RAF)

```



```
| └─ HistoryScreenContent (heavy - 1875 lines, ~56 hooks)
|   └─ Conversations tab      Thread list with swipe-delete
|   └─ Emotion History tab    Mood timeline with AppChip badges
|   └─ AppChip                Status pills (theme-aware all 5 variants)
|
| └─ TrendsScreen (shell - useFocusEffect + double-RAF)
|   └─ TrendsScreenContent (~43 hooks)
|     └─ FeatureDiscoveryCard  194-tip hourly rotation (active-time)
|     └─ PendingInsights section weeklyRecap / pulse / milestone / companionInsight
|     └─ FeelSection           Quick mood log (8 emotion buckets)
|     └─ LettersFromImotara    Letter archive with TTS / reactions / replies
|     └─ CompanionLetterCard   Individual letter with listen/react/reply
|     └─ 30-day Challenge      Grid dot progress tracker
|     └─ Emotional Fingerprint Abstract emotion pattern visualisation
|     └─ Reflection Journal    Private entries (local only)
|
| └─ SettingsScreen (shell - useFocusEffect + double-RAF + isDark detection)
|   └─ SettingsScreenContent (heavy - ~3200 lines, ~140 hooks)
|     └─ SettingsSearch        AI-powered settings finder (local + /api/settings-search)
|     └─ Accordion: Plan & support Upgrade, donate, restore
|     └─ Accordion: Your plan   Tier display, token balance
|     └─ Accordion: Experience  Theme, TTS, voice, reactions, tips, sync
|     └─ Accordion: Your companion Name, tone, gender, age, letter cadence
|     └─ Accordion: Privacy & safety Export, clear, delete account, backup
|     └─ Accordion: Mindset Analysis Period toggles (today/7d/30d/all)
|     └─ Accordion: Advanced    Memory, debug, version info
```

9. Theme System & Color Palette

9.1 Color Palette (DARK / LIGHT)

Token	Dark Mode	Light Mode	Usage
background	rgba(19,14,23,1)	rgba(248,250,252,1)	Screen backgrounds
surface	rgba(24,15,30,0.9)	rgba(241,245,249,0.95)	Cards, modals
surfaceSoft	rgba(24,15,30,0.7)	rgba(226,232,240,0.85)	Chips, inputs, subtle bg
border	rgba(148,163,184,0.25)	rgba(100,116,139,0.20)	Borders, dividers
textPrimary	rgba(241,245,249,1)	rgba(15,23,42,1)	Main body text
textSecondary	rgba(148,163,184,0.9)	rgba(71,85,105,0.9)	Secondary/hint text
primary	rgba(56,189,248,1)	rgba(14,165,233,1)	Accent, buttons, links
primaryTint	rgba(56,189,248,0.15)	rgba(14,165,233,0.12)	Tinted backgrounds

9.2 Theme Architecture

ThemeContext exposes `useTheme() → { colors, isDark }` and `useColors() → ColorPalette`. The theme mode is persisted to `AsyncStorage (mobile) / localStorage (web)`. All screens subscribe to `ThemeContext` via hooks.

Blank screen prevention: Heavy screens (Settings 140 hooks, History 56, Trends 43) use a double-RAF pattern on `isDark` changes — they briefly unmount content (shows spinner), then remount after 350ms + 2 animation frames, preventing JS thread freezes during theme switches.

10. Navigation Architecture

```
NavigationContainer (deep-link: imotara://)
└─ createBottomTabNavigator
    ├── Chat      (headerShown: false) → tabBarIcon: chatbubble-ellipses
    ├── History   (title: "History")   → tabBarIcon: time
    ├── Trends    (title: "Trends")    → tabBarIcon: bar-chart
    │           → tabBarBadge: pendingInsights count
    │           → badge auto-clears when tab focused
    └─ Settings   (title: "Settings") → tabBarIcon: settings

Deep Link Config:
imotara://chat      → Chat tab
imotara://history   → History tab
imotara://trends    → Trends tab
imotara://settings → Settings tab

Tab bar styling:
backgroundColor: colors.surfaceSoft
borderTopColor: colors.border
activeTintColor: colors.primary
animation: "fade"
tabBarHideOnKeyboard: true
```

11. AI / Psychological Engine

11.1 MASTER FRAMEWORK — 7-Tier Priority Ladder

Tier	Name	Triggers	Tools
1	CRISIS / SAFETY / SHAME	Flood, self-attack, hopelessness, rupture	S1 TIPP, S6 Grounding, PA6 Inner Critic, T5 Hope, X8 Repair, Z6 Harm Reduction
2	ESTABLISH UNDERSTANDING	First pain mention, vague emotion, caregiver	M1 Intensity, M3 Distress Thermometer, Z2 Affect Label, X12 Compassion Fatigue

3	DEEPEN UNDERSTANDING	Anger, conflict, ongoing anxiety, people-pleasing	T2 Secondary Emotion, T3 Parts Work, M2 Functional Impact, S3 Fawn, PA1 Attachment
4	SEE THE PATTERN	"Always/never", repeated pain, disproportionate feelings	T1 Cognitive Distortion, PA5 Repetition Compulsion, PA4 Transference, T8 Core Belief, Z5 Intergenerational
5	OFFER A NEW ANGLE	Fixed self-story, stuck, identity fused with problem	T9 Narrative Re-Authoring, T10 Miracle Question, Z3 PTG, PA3 Shadow, T3/X4 Parts/EFT
6	MOVE TOWARD CHANGE	Waiting for motivation, harmful emotion, chronic worry	T12 Behavioral Activation, Z1 Opposite Action, X3+X11 Metacognitive, X9 DEAR MAN, T4 Values
7	INTEGRATE & CELEBRATE	Positive/brave, small wins, conversation ending	S9 Celebration, X10 Savoring, M4 Progress Marker, M5 Session Effectiveness, Z3 PTG

11.2 Tool Categories (71 total)

T1-T12 Core Therapeutic (CBT, ACT, narrative, DBT, grief, behavioral activation)	PA1-PA12 Psychoanalytic (attachment, inner child, shadow, transference, repetition compulsion)	M1-M5 Measurement (intensity probe, functional impact, distress thermometer, progress marker)
S1-S9 Specialized (TIPP, mindfulness, fawn, self-compassion, polyvagal, grounding, celebration)	X1-X12 Extended (motivational interviewing, EFT, metacognitive, schema, DEAR MAN, compassion fatigue)	Z1-Z6 Final Six (opposite action, affect labeling, PTG, RSD, intergenerational, harm reduction)
W1-W6 Final Additions (Drama Triangle, SDT, mentalizing, radical acceptance, grief for unlived life, existential loneliness)		

12. Payment & Licensing Architecture

12.1 Product Catalog

Product ID	Type	Tier	Duration	Price (INR)
plus_monthly	Subscription	Plus	31 days	₹99
plus_annual	Subscription	Plus	366 days	₹699
pro_monthly	Subscription	Pro	31 days	₹149
pro_annual	Subscription	Pro	366 days	₹1,299
tokens_100	Token Pack	—	One-time	₹49

tokens_250	Token Pack	—	One-time	₹99
tokens_600	Token Pack	—	One-time	₹199
tokens_1800	Token Pack	—	One-time	₹499

12.2 Android Payment Flow (Razorpay)

```

Mobile → POST /api/license/order-intent { productId }
    ↓ Creates Razorpay Order (amount, notes: {productId, userId})
Mobile ← { orderId, keyId, amount }
    ↓ Opens RazorpayCheckout.open()
    ↓ User pays (UPI, card, NetBanking, UPI mandate)
Mobile gets razorpay_payment_id
Mobile → POST /api/license/verify-payment { paymentId, productId }
    ↓ Polls Razorpay API up to 5× (2s each) for "authorized"→"captured"
    ↓ Accepts "authorized" (UPI mandate pre-capture)
    ↓ grantLicense(userId, productId, source:"razorpay")
Mobile ← { ok: true, tier, tokenBalance }

Webhook fallback:
Razorpay → POST /api/payments/razorpay/webhook (HMAC-verified)
    ↓ event: payment.captured OR order.paid
    ↓ Reads notes.productId, notes.userId from payment/order entity
    ↓ grantLicense(userId, productId, source:"webhook")

```

12.3 iOS Payment Flow (Apple IAP / StoreKit 2)

```

useIAP({ onPurchaseSuccess }) - expo-iap StoreKit 2
    ↓ User taps → requestPurchase({ type, request: { apple: { sku } } })
    ↓ onPurchaseSuccess fires with purchase.transactionId
Mobile → POST /api/license/verify-apple-purchase { productId, transactionId }
    ↓ Fetches JWS token from Apple API
    ↓ Verifies ES256 signature (dsaEncoding: ieee-p1363)
    ↓ Retries 3× on 404 (StoreKit race condition)
    ↓ grantLicense(userId, productId, source:"apple")
Mobile ← { ok: true, tier }
    ↓ finishTransaction({ purchase, isConsumable })

```

13. Feature Architecture — Chat Screen

13.1 Clean Chat Interface Design

The chat screen was redesigned to show only conversation-essential elements. Non-conversational insight cards moved to the Trends tab via an AsyncStorage bridge (pendingInsights.ts).

Element	Location	Trigger
Pending undo toast	Chat	5s after send
Crisis resources	Chat	Safety signals in message
Open loop revisit	Chat	Unresolved theme detected
Unsent letter hint	Chat	Relationship keywords detected
Daily check-in chips	Chat	Once per day
Sentiment chips	Chat	Input empty, togglable
Reflection seed cards	Chat (inline)	After AI reply
Tone reflection card	Chat (inline)	End of meaningful session
Weekly mood recap	→ Trends	Moved — via pendingInsights
Collective pulse	→ Trends	Moved — via pendingInsights
Milestone celebration	→ Trends	Moved — via pendingInsights
Companion insight	→ Trends	Moved — via pendingInsights
Grow nudge	Removed	Tab bar is always visible

13.2 Voice Pipeline

```

User taps mic → useVoiceInput hook
  ↓ expo-av recording starts
  ↓ FileSystem.uploadAsync() [Hermes-safe, production-reliable]
  ↓ POST /api/voice/transcribe (multipart)
  ↓ OpenAI Whisper API (55s timeout)
  ↓ Transcript returned → shown in input bar
  ↓ User can edit before sending

TTS (Text-to-Speech):
mobileTTS.ts → checks native voice availability
If native: expo-speech.speak() [free, offline]
If Azure: POST /api/tts → Azure Neural TTS → MP3 → expo-av play
Hands-free: auto-speak + auto-listen after each reply

```

14. Feature Architecture — Trends Screen

14.1 Trends Screen Content Order

```

TrendsScreen (scrollable)
├── FeatureDiscoveryCard          194 tips, 30-min active time rotation
│   ├── Active time: AppState listener tracks foreground/background
│   ├── flushActiveTime() on background → accumulated_ms persisted
│   └── getCurrentTip() adds live session time before checking threshold
├── Pending Insights (from Chat)
│   ├── CompanionInsightCard     Letter or emotional arc (if pending)
│   ├── Milestone card           "You closed a loop +"  

│   ├── Weekly recap             7-day emotional theme summary  

│   └── Collective pulse          % of users carrying something heavy
├── FeelSection                  Quick emotion log (8 buckets)
├── Streak card                  Consecutive days chatted
├── Weekly mood chart            Emotion frequency bar chart
├── LettersFromImotara           Letter archive (listen / react / reply)
├── 30-day Challenge             Dot grid progress + daily prompt
├── Emotional Fingerprint        Abstract emotion pattern visualisation
├── Reflection Journal           Private local entries
├── On This Day                  Same date in past months
└── Emotional Arc                Monthly narrative story

```

14.2 Companion Letter Generation

```

generateCompanionLetter(threads, profile)
├─ buildDeepPsychProfile(threads)
│   ├── Schema signals: defectiveness, abandonment, entrapment, shame, self-blame
│   ├── Growth signals: realization, acceptance, boundary-setting, gratitude
│   ├── Relational signals: childhood, romance, loneliness, conflict, grief
│   ├── Language-agnostic quotes: first 4 messages > 20 chars (no English filter)
│   └── Raw sample: last 10 messages in original language (all 22 languages)
├─ Prompt with: relationship depth hint, psychological observations, 7 writing rules
│   1. WITNESS: specific observation
│   2. CORE WOUNDS: warm plain language (not clinical)
│   3. GROWTH: as beginning, not completion
│   4. GENUINE ADMIRATION: specific evidence
│   5. AFFECT PRECISION: exact emotion language
│   6. CLOSE WITH LIGHT: honest possible hope
│   7. MYTH/POETRY: optional if fits naturally
├─ POST /api/respond (isLetterMode: true)
└─ Saved to archive (max 24 letters, ~2 years)

```

15. Performance & Production Patterns

15.1 Blank Screen Prevention (Heavy Screens)

Problem: Settings (140 hooks), History (56), Trends (43) caused blank screens on:

- Tab switch (navigation animation incomplete)
- Theme switch (ThemeContext re-render cascade freezes JS thread)

Solution - Double-RAF pattern:

```
useFocusEffect(useCallback(() => {
  setScreenReady(false); // ← unmount heavy content, show spinner
  const timer = setTimeout(() => {
    requestAnimationFrame(() => { // ← RAF1: schedule spinner frame
      requestAnimationFrame(() => { // ← RAF2: spinner committed to screen
        setScreenReady(true); // ← NOW mount heavy content
      });
    });
  }, 300); // ← wait for nav animation
  return () => { clearTimeout(timer); setScreenReady(false); };
}, []));

useEffect(() => { // ← isDark change detection
  if (prevIsDarkRef.current === isDark) return;
  prevIsDarkRef.current = isDark;
  setScreenReady(false);
  themeTimerRef.current = setTimeout(() => {
    requestAnimationFrame(() => requestAnimationFrame(() => setScreenReady(true)));
  }, 350);
}, [isDark]);
```

15.2 Network Reliability (Hermes Production)

Pattern	Problem Solved	Implementation
fetchWithTimeout()	AbortSignal.timeout() not in Hermes; bare fetch hangs on mobile networks	AbortController + setTimeout wrapper — used on ALL fetch calls
FileSystem.uploadAsync	FormData.append({uri}) silently corrupts audio files in production Hermes	expo-file-system/legacy MULTIPART upload for voice transcription
Payment polling	UPI mandate payments are "authorized" not "captured" immediately	verify-payment polls Razorpay 5× / 2s; accepts "authorized" + webhook confirms
Active-time tracking	Wall-clock feature tips fired while app was closed	AppState listener accumulates ms only while foregrounded; persisted to AsyncStorage

15.3 Settings Search — Hybrid Approach

```
User types query
↓
Local fuzzy search (instant, offline)
├─ Scores each of 37 settings across: title, keywords, description, section
├─ Returns top 5 matches with score
├─ If top score ≥ 8 AND query < 3 words → show results immediately

If low confidence (score < 8) OR sentence query (≥ 3 words):
→ POST /api/settings-search { query }
```

```
→ System prompt: 37 settings catalog + "any of 22 languages"
→ AI returns { ids: ["setting_id_1", "setting_id_2"] }
→ Map IDs to catalog entries → display results
```

On result tap:

```
→ Open accordion section (sectionMap[sectionKey])
→ setTimeout(350ms) for accordion animation
→ RAF1 + RAF2 → scroll to stored Y position of section header
```

16. Key Design Decisions

Local-First Storage

All user conversations, settings, journal, and emotional data are stored on-device (AsyncStorage mobile / localStorage web) by default. Cloud backup is opt-in only via explicit sign-in. This builds user trust and works without internet.

Pending Insights Bridge (Chat → Trends)

Weekly recap, collective pulse, milestone, and companion insight cards were removed from the Chat screen to reduce clutter. They now live in Trends via an AsyncStorage bridge (pendingInsights.ts). Chat writes; Trends reads on focus. A badge dot on the Trends tab signals unseen content.

No "AI" Branding in UI

All user-facing text avoids "AI" to prevent users feeling like they're talking to a chatbot. "Cloud sync" → "Account backup". "AI search" → "✨ (sparkle)". The companion responds as a personal relationship, not a product.

Theme-Aware Components

All components use `useTheme().isDark` or `useColors()`. No hardcoded dark-mode values (rgba(255,255,255,0.04) etc.) in user-visible elements. AppChip, DiscoveryCard, UnsentedLetterModal, BreathingModal, HistoryScreen sync chip — all converted to dynamic palette selection.

UPI Mandate Payment Reliability

UPI mandate payments land as "authorized" (not "captured") immediately after user payment. verify-payment now polls Razorpay 5x with 2s delay. If still authorized, license is granted (idempotent). Webhook fires "payment.captured" within seconds as confirmation. Mobile polls /api/license/status for 21s as additional fallback.

22-Language Support Architecture

The AI system prompt handles all 22 languages natively. For features requiring language detection (companion letters, mindset analysis, emotional arc), regex pattern matching handles romanized scripts, while raw message samples are passed to the AI for non-Latin scripts (Bengali, Arabic, Chinese, Japanese, etc.) which the AI reads natively.

Imotara v1.1.9 · Build 97 · Web · iOS · Android · May 2026
Private Emotional Wellness Companion — imotara.com

Imotara System Design Document — Internal Reference · v1.1.9
Architecture, data models, and design decisions as of May 2026.
Not for public distribution.